TRANSFORMED 2 SUCCEED

transformed2succeed.com

FREE LIVE SESSION · 30 MINUTES

SRE for AI-ML Systems

Measure what users feel, alert only when it matters, and handle the 2 a.m. page calmly.

PRESENTER Dr. Emmanuel Naweji **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of SRE for AI-ML Systems, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

THE ONE IDEA

Measure what users feel. Alert on burn. Stop the bleeding first.

01

If users feel it, measure it

Up, fast and right. Not CPU, not 'the server is up'.

02

Alert on burn, not blips

Turn the SLO into a monthly budget, and page only when it burns fast.

03

Stop the bleeding first

Roll back, verify, then learn with a blameless postmortem.

COURSE MAP

Three modules, one calm on-call

MODULE 1

Measure What Users Feel

Define SLIs and SLOs, see why averages lie, and catch a model that is up but wrong.

MODULE 2

Spend the Error Budget

Turn the SLO into a monthly budget and build alerts that page only on real burns.

MODULE 3

When It Breaks

Run a game day: two incidents, two rollbacks and a blameless postmortem.

01

MODULE 1 OF 3

Measure What Users Feel

A model can be up, fast… and wrong. If users feel it, measure it.

Every server metric green. Users waiting.

30%

CPU: the server is relaxed

100%

up: the process never stopped

175 ms

average latency: looks fine

30%

of users waited 0.6 s or failed

Numbers from the lab's v2 model. Servers measure themselves; SLIs measure what users feel.

Three things users feel, three targets

01 **Up**

Availability: at least 99.5% of requests succeed.

02 **Fast**

Latency: at least 95% of requests answer in under 300 ms.

03 **Right**

Canary accuracy: at least 90% on known answers. The AI part.

04 **Never 100%**

100% means no deploys, no risk, no progress. Users can't tell anyway.

SLI = the measure. SLO = the target. SLA = a contract with money attached.

Averages lie

THE AVERAGE

mean 175 ms: “looks fine”

Adds everyone up and divides. Fast users hide the slow ones.

VS

THE 95TH PERCENTILE

p95 602 ms

Sort all requests: 95% were faster, 5% slower. 1 in 20 users waited 0.6 s or more.

Latency SLOs use percentiles (p95, p99) or 'share of requests under X ms'. Never the average.

Ship a wrong model. Watch only one SLI notice.

TERMINAL

```
$ python3 deploy.py v3
$ python3 load.py --requests 300 && python3 sli.py --last 300
```

`deploy.py v3`
switch the live model; logged to deploys.log

`load.py`
300 pretend users, 1 in 5 a canary with a known answer

`sli.py --last 300`
score only the newest 300 requests

`v3`
a bad retrain: fast, confident, labels every ticket 'normal'

YOU SHOULD SEE PASS availability 100.00% · PASS requests <= 300 ms 100.0% · FAIL canary accuracy 53.3% (yours will vary)

Your lab: measure a live model service

1

Start the service

start.sh gives a fresh ticket-triage service; server.py logs every request.

2

Measure healthy v1

Send 300 requests, run sli.py: up, fast and right. SLOs MET.

3

Break it twice

v2 is slow and crashes; v3 is up, fast and wrong. See which SLI catches each.

4

Roll back and verify

deploy.py v1, fresh traffic, SLOs MET again. Then stop with kill.



A model can be up, fast… and wrong.

Regular services stop at two SLIs. AI services need the third: is it right?

02

MODULE 2 OF 3

Spend the Error Budget

100% is the wrong target. Alert on burn, not on blips.

Paged 34 times for nothing

34

pages from 'error rate > 5% in any minute'

40

one-minute blips: normal life

22%

of the error budget used

0

real incidents

Real output from budget.py on the 'quiet' month. The 35th page is the one people ignore.

Your SLO, turned into minutes

7.2 h

of failure allowed per 30 days at 99%

216 min

at 99.5%: our SLO (≈ 3.6 hours)

43 min

at 99.9%: each extra 9 costs about 10x

Budget = (1 − SLO) × the window. 0.5% of 43,200 minutes = 216 minutes. Pick the number users need.

Burn rate: how fast are you spending?

1x

lasts 30 days: exactly on budget. Fine.

2.4x

≈ 12 days: the slow leak. Ticket.

14.4x

≈ 2 days: 2% of the month in 1 hour. Page.

25x

≈ 29 hours: the bad deploy. Page now.

Burn rate = error rate now ÷ error rate the SLO allows. Example: 12.6% ÷ 0.5% ≈ 25x.

Two windows, one alert

01

Long window: is it real?

A one-minute blip can't push a 1-hour burn rate over 14.4x. Noise filtered.

02

Short window: still on?

Once you roll back, the 5-minute rate drops fast, so the page clears fast.

03

Both must agree

Page only when the long AND short windows are over the line.

PAGE: 1h & 5m > 14.4x, or 6h & 30m > 6x. TICKET: 3d & 6h > 1x. Otherwise: sleep.

A bad deploy in the last 40 minutes

TERMINAL

```
$ python3 simulate.py bad-deploy && python3 budget.py
```

simulate.py
30 days of per-minute traffic, fixed seed: 43,200 rows

bad-deploy
a normal month, then the last 40 minutes at 20% bad

budget.py
budget, burn rates in 5 windows, PAGE / TICKET / OK

PAGE
1h and 5m are both over 14.4x

YOU SHOULD SEE used 23% · burn now: 5m 30.5x · 30m 37.8x · 1h 25.3x · 6h 3.0x · 3d 0.4x · PAGE fast burn

Your lab: alert like an SRE

1

A quiet month

22% used, burn 0.1–0.2x: OK. The naive rule fired 34 times.

2

A bad deploy

The burn rate explodes: PAGE.

3

A slow leak

3 days at 1.2% bad: 43% used, about 2.4x. TICKET, nobody woken.

4

Tighten the SLO

Same quiet month at 99.9%: 109% used. Every 9 has a price.



Alert on burn, not on blips.

Every page should be urgent, real and actionable.

03

MODULE 3 OF 3

When It Breaks

The page has fired. Users are hurting. Stop the bleeding first.

Fixed in one minute. Found after forty-four.

02:07

fast-burn page fires

44 min

spent finding the root cause

1 min

the rollback took, once tried

02:52

users finally OK

A composite story. The deploy at 01:58 was the cause; rolling it back first would have ended the pain in minutes.

The first five minutes

1

Acknowledge

Tell the pager you've got it, so it stops escalating.

2

Confirm

python3 sli.py --last 300: which SLO is failing, and how badly?

3

Declare

Open an incident channel: impact, who's leading, next update.

4

What changed?

python3 deploy.py --history. A recent deploy is suspect #1.

Undo first. Fix forward later.

FIX FORWARD

debug → patch → deploy

Find the bug at 2 a.m., write new code, ship it untested. Slow, and risky twice over.

VS

ROLL BACK

python3 deploy.py v1

Go back to the last version that met its SLOs. Fast, known-good, boring. Perfect.

Rollback works only if it's one command and the old version is still ready.

When the model is wrong, not down

01

Roll back the model

Serve the previous model version. Keep the last good one ready.

02

Fall back

Simpler rules or an older model for risky cases. Worse, but safe.

03

Gate the next one

Canary deploys and a quality gate, so it never reaches all users again.

Incident 2: everything looks fine…

TERMINAL

```
$ bash .lab/incident.sh 2
$ python3 sli.py --last 300
$ python3 deploy.py --history
$ python3 deploy.py v1 && python3 load.py --requests 300
```

`incident.sh 2`
a 'teammate' ships something, then pages you

`sli.py`
runbook step 1: confirm the pain

`--history`
step 2: what changed? v1 -> v3, minutes ago

`deploy.py v1`
step 3: roll back, then verify with fresh traffic

YOU SHOULD SEE PAGER … · PASS availability · PASS requests <= 300 ms · FAIL canary accuracy ~45–60%

Your lab: game day, you're on call

1

Record a healthy baseline

Start the service, send traffic: SLOs MET.

2

Incident 1: slow + crashes

Follow RUNBOOK.md: confirm, what changed, roll back, verify.

3

Incident 2: up but wrong

Only canary accuracy fails. Same runbook, same rollback.

4

Write the postmortem

Blameless: impact, timeline, root cause, owned action items.



Stop the bleeding first. Understand it second.

A tourniquet before the X-ray.

RECAP

Three modules. Three phrases to keep.

If users feel it, measure it

Up, fast, right. Percentiles, not averages. A quality SLI for the model.

Alert on burn, not on blips

Budget = (1 − SLO) × window. Two windows must agree. Page fast, ticket slow.

Stop the bleeding first

Confirm, what changed, roll back, verify. Then a blameless postmortem.



You can handle the 2 a.m. page calmly.

Measure what users feel · Alert on burn · Stop the bleeding first

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- DevOps for AI-ML Systems
- **SRE for AI-ML Systems**
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	Linux for AI-ML Systems	Zero to AI-ML Systems Engineering	TONIGHT SRE for AI-ML Systems	FinOps for AI-ML Systems
THURSDAY	Python for AI-ML	DevOps for AI-ML Systems	NEXT GPU Optimization for AI-ML Systems	AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Thursday, 7:30 PM Central · GPU Optimization for AI-ML Systems

Questions? Let's take them now.

Dr. Emmanuel Naweji · Transformed 2 Succeed